Complaint System Bug Report

Focused review of confirmed correctness, security, and maintainability issues

Repository: complaint-system

Review date: 6 September 2026

Scope: Express/MongoDB backend and React frontend

Validation: Frontend lint and production build passed

7	1	5.5/10
confirmed issues	critical security issue	prototype rating

Executive Summary

The application has a clear full-stack structure and the frontend compiles cleanly. The main risk is authorization: authenticated users can provide identity fields that the server should derive from the verified JWT. The project also lacks backend tests, upload restrictions, robust validation, and atomic data handling for likes and comments.

Confirmed Bugs

BUG-001: User identity is trusted from request data CRITICAL

Affected files: `backend/src/controller/complaint.js`, `complaint-frontend/src/components/createComplaint/CreateComplaint.jsx`

Problem: Complaint creation reads `userId` from `req.body`, and comment creation also accepts `userId` from the client. The JWT has already been verified, but its identity is not used for these writes.

Impact: Any authenticated user can create a complaint or comment under another user's account by changing the submitted ID. This can also undermine ownership checks and profile data.

Fix: Use `req.user.userId` on the server. Remove `userId` from the client payload and request validation.

BUG-002: Comments are not linked back to complaints HIGH

Affected file: `backend/src/controller/complaint.js`

Problem: A `Comment` document is created, but its ID is never added to the parent complaint's `comments` array.

Impact: Complaint queries using `populate("comments")` may return no comments even after successful comment creation.

Fix: Update the complaint after creation, or redesign the relationship to query comments by `complaintId` consistently.

BUG-003: File uploads have no size or type restrictions HIGH

Affected file: `backend/src/util/multer.js`

Problem: Multer uses memory storage with no `limits` and no file filter. The server accepts arbitrary files and loads them fully into memory before forwarding them to Cloudinary.

Impact: Oversized requests can cause memory pressure or denial of service, and unexpected file types can be stored as evidence images.

Fix: Restrict MIME types to supported image formats, set a strict byte limit, and validate content where appropriate.

BUG-004: JWT uses a predictable fallback secret MEDIUM

Affected file: `backend/src/util/jwt.js`

Problem: Missing environment configuration silently falls back to `complaintbox-secret`.

Impact: A deployment mistake can make token forgery possible for anyone who knows the source code.

Fix: Fail fast during startup when `JWT_SECRET` is absent or too weak. Keep secrets outside the repository.

BUG-005: Like toggling is not atomic or duplicate-safe MEDIUM

Affected files: `backend/src/controller/complaint.js`, `backend/src/models/like.js`

Problem: The endpoint checks for an existing like, then creates or deletes it and separately increments or decrements the complaint counter. The schema has no compound unique index.

Impact: Concurrent requests can create duplicate likes or leave the counter inconsistent, including possible negative counts.

Fix: Add a unique compound index on `{ userId, complaintId }` and use an atomic update or transaction.

BUG-006: Backend has weak validation and inconsistent HTTP errors MEDIUM

Affected files: backend controllers and routes

Problem: Several authentication failures return JSON with HTTP 200. There is no centralized error middleware, schema validation layer, rate limiting, or backend integration test suite.

Impact: Clients and monitoring systems cannot reliably distinguish successful requests from failures, while malformed or abusive input receives limited protection.

Fix: Return appropriate 4xx/5xx status codes, validate request bodies, centralize error handling, rate-limit authentication, and add API tests.

BUG-007: Duplicate authentication page implementations MEDIUM

Affected files: `complaint-frontend/src/components/AuthPage/AuthPage.jsx`, `complaint-frontend/src/pages/AuthPage/AuthPage.jsx`

Problem: Two separate `AuthPage` components implement the same login/register workflow with different markup and behavior.

Impact: Future fixes can be applied to one version only, producing inconsistent authentication UX and unnecessary maintenance.

Fix: Keep one canonical component and remove or rename the unused duplicate.

Recommended Fix Order

1. Derive all write-operation identities from the JWT.
2. Add upload size/type restrictions.
3. Repair comment persistence and add the like uniqueness constraint.
4. Require a strong JWT secret and improve HTTP error semantics.
5. Add backend tests for authentication, ownership, comments, likes, and uploads.
6. Remove the duplicate authentication component.

Generated from the code review of the complaint-system repository. Frontend checks observed: ESLint passed; Vite production build passed.